

FOLIO

Stack Técnica — Documento Vivo

v1.0 • Documento em evolução — atualizado conforme o projeto avança

1. Visão Geral

Este documento registra as tecnologias escolhidas para o desenvolvimento do Folio. Ele não é estático — novas tecnologias serão adicionadas conforme o projeto avança e novas necessidades emergem. Cada escolha tem uma justificativa e um status claro.

Critérios que guiaram as escolhas:

- Alinhamento com a stack que já está sendo estudada (TypeScript, Node.js, Next.js)
- Capacidade de escalar para um produto real sem reescritas dolorosas
- Ecossistema maduro e documentação sólida
- Custo zero ou baixo na fase inicial (MVP)

2. Stack Atual

Tecnologias confirmadas para o MVP. Todas as entradas com status "A avaliar" já estão no radar, mas aguardam validação de necessidade real antes de serem incorporadas.

Tecnologia	Camada	Função	Status
TypeScript	Transversal	Tipagem estática em todo o projeto (front e back)	Definido
Node.js	Backend	Runtime do servidor	Definido
Express	Backend	Framework HTTP — rotas, middlewares, controllers	Definido
Next.js 14	Frontend	App Router, SSR/SSG, deploy automático na Vercel	Definido
PostgreSQL	Banco	Banco relacional principal	Definido
Prisma	Banco	ORM type-safe, migrations automáticas	Definido
Vitest	Testes	Testes unitários e de integração (backend e frontend)	Definido

Cypress	Testes	Testes end-to-end — fluxos completos da aplicação	Definido
Cache (Redis)	Infra	Cache de sessões, rate limiting, dados frequentes	A avaliar
Mensageria	Infra	Filas de processamento assíncrono (ex: BullMQ + Redis)	A avaliar

3. Detalhamento por Camada

3.1 Linguagem base

TypeScript é o elo que conecta toda a stack. Tanto o backend (Node.js + Express) quanto o frontend (Next.js) rodam em TypeScript, o que garante:

- Contratos claros entre frontend e backend via tipos compartilhados
- Detecção de erros em tempo de desenvolvimento, não em produção
- Integração nativa com Prisma — o ORM já gera tipos a partir do schema

3.2 Backend

Node.js + Express

Escolha direta: amplamente adotado no mercado, excelente fit com TypeScript, e já faz parte da stack de estudo. Express foi preferido sobre Fastify ou NestJS por ser mais explícito — menos magia, mais controle, mais fácil de debugar.

Estrutura de pastas do backend (folio-api/):

- controllers/ — recebe requisições HTTP e delega para services
- services/ — lógica de negócio
- repositories/ — acesso ao banco via Prisma
- middlewares/ — autenticação JWT, validação de payload
- routes/ — definição das rotas

3.3 Frontend

Next.js 14 com App Router. Escolhido por ser o framework React com maior adoção no mercado e suporte nativo a SSR/SSG — importante para SEO da landing page e das páginas públicas. O frontend é deployado automaticamente na Vercel a cada push.

3.4 Banco de dados e ORM

PostgreSQL é o banco relacional principal. Para o MVP, será provisionado via Supabase (plano gratuito generoso, sem necessidade de gerenciar servidor).

Prisma atua como ORM: gera tipos TypeScript diretamente do schema, gerencia migrations e abstrai queries com type-safety completo. O workflow é:

- Editar `schema.prisma` → rodar `prisma migrate` → types gerados automaticamente

3.5 Testes

Vitest — testes unitários e de integração. Escolhido sobre Jest pela performance superior em projetos TypeScript e compatibilidade nativa com Vite/Next.js. Cobre:

- Funções de negócio isoladas (services, utils)
- Endpoints da API (integração com banco de teste)

Cypress — testes end-to-end. Simula o usuário real navegando na aplicação. Cobre:

- Fluxos críticos: cadastro, login, criação de entradas no Daily Log, marcação de hábitos
- Regressão visual após mudanças de UI

3.6 Cache (a avaliar)

Redis é a tecnologia mais provável para cache quando a necessidade surgir. Casos de uso previstos:

- Cache de sessões de usuário (alternativa a JWT stateless)
- Rate limiting em endpoints sensíveis (login, criação de contas)
- Cache de dados lidos com frequência (ex: lista de plugins disponíveis)

Decisão: será adicionado ao projeto quando o volume de dados ou requisições justificar. Não será adicionado preventivamente.

3.7 Mensageria (a avaliar)

BullMQ sobre Redis é a combinação mais provável se processamento assíncrono for necessário. Casos de uso previstos:

- Envio de e-mails transacionais em fila (boas-vindas, notificações de revisão)
- Processamento de dados de plugins sem bloquear a requisição principal
- Jobs agendados (ex: gerar o resumo semanal do usuário)

Decisão: será adicionado quando um caso de uso concreto surgir no desenvolvimento. Não entra no MVP.

4. Autenticação

JWT com refresh token — implementação própria, sem biblioteca de autenticação externa.
Escolhido por:

- Controle total sobre o fluxo de autenticação
- Sem dependência de serviço de terceiros que pode mudar preço ou política
- Exercício direto dos conceitos de segurança (assinatura, expiração, rotação de tokens)

Fluxo básico:

- Access token: curta duração (15min), enviado no header Authorization
- Refresh token: longa duração (7 dias), armazenado em cookie httpOnly
- Rota /auth/refresh troca o refresh token por um novo par de tokens

5. Hospedagem e Deploy

Tecnologia	Camada	Função	Status
Vercel	Frontend	Deploy automático do Next.js a cada push no GitHub	Definido
Railway	Backend	API Node.js + Express — simples, sem gerenciar servidor	Definido
Supabase	Banco	PostgreSQL gerenciado — plano free para MVP	Definido
GitHub Actions	CI/CD	Pipeline de build e testes automáticos	Definido
Resend	Email	Envio de e-mails transacionais	Definido

6. Log de Atualizações

Registro de mudanças na stack ao longo do projeto.

Data	Mudança	Justificativa
Jun 2025	Stack inicial definida	Documento criado com as tecnologias do MVP.
—	—	—
—	—	—

—	—	—
---	---	---

Folio — construído com método, sem impor nenhum.